

INTELLIGENT API ATTACK DETECTION AND RUNTIME PROTECTION FRAMEWORK

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

ITA1406- ETHICAL HACKING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

A.K Selva Prabhu (192421224)

M.S. Sanjay Josewa(192511333)

E. Hariharan (192421057)

Under the Supervision of

Dr. A Ezhil grace



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST- 2026



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

We **A.K Selva Prabhu , M.S. Sanjay Josewa, E. Hariharan** of Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled '**Community Issue Reporting App with Geo-Tagged Media Uploads**' is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date:

Signature of the Students with Names

A.K Selva Prabhu (192421224)

M.S. Sanjay Josewa(192511333)

E. Hariharan (192421057)



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Intelligent Api Attack Detection And Runtime Protection Framework**” has been carried out by **A.K Selva Prabhu , M.S. Sanjay Josewa, E. Hariharan** under the supervision of **Dr A. Ezhil grace** and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech CSE program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr A Ezhil grace

Department of CSE

Saveetha School of Engineering

SIMATS

SIGNATURE

Dr. Sasi Rekha

Program Director

Department of CSE

Saveetha School of Engineering

SIMATS

Submitted for the Project work Viva-Voce held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Prof. Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, **Dr. Sasi Rekha** Kumar for his continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **Dr. A. Ezhil grace** for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

A.K Selva Prabhu (192421224)

M.S. Sanjay Josewa(192511333)

E. Hariharan (192421057)

ABSTRACT

Modern software applications heavily depend on Application Programming Interfaces (APIs) for communication between web applications, mobile applications, cloud services, and backend systems. As APIs expose important data and business functions, they have become a major target for cyberattacks. Traditional API security mechanisms such as authentication, authorization, firewalls, and predefined security rules may not be sufficient to detect evolving and unknown attack patterns in real time.

The **Intelligent API Attack Detection and Runtime Protection Framework** is a security solution designed to monitor API traffic, identify malicious activities, and automatically protect APIs during runtime. The framework continuously analyzes API requests and responses using security rules, behavioural analysis, anomaly detection, and machine learning techniques. It can identify suspicious activities such as repeated authentication failures, excessive requests, injection attempts, unauthorized access, abnormal API usage, and other malicious behaviours.

The framework consists of an **API Traffic Monitoring Module, Intelligent Attack Detection Module, Runtime Protection & Response Module, Security Analytics Dashboard, and Security Data Management Module**. When suspicious activity is detected, the system can automatically block requests, apply rate limiting, generate alerts, or temporarily restrict suspicious clients. Administrators can monitor API traffic, detected attacks, severity levels, affected endpoints, and response actions through a centralized dashboard.

The proposed framework improves API security by providing continuous monitoring, intelligent attack detection, automated runtime protection, faster incident response, and improved API availability.

Keywords: API Security, Attack Detection, Runtime Protection, REST API, JWT, OAuth, Machine Learning, Anomaly Detection, Rate Limiting, Cybersecurity.

TABLE OF CONTENTS

Chapter	Contents	Page No.
Chapter 1:	Introduction	8
	1.1 Background Information	9
	1.2 Project Objectives	9
	1.3 Significance	9
	1.4 Scope	9
	1.5 Methodology Overview	10
Chapter 2:	Problem Identification and Analysis	11
	2.1 Description of the Problem	11
	2.2 Evidence of the Problem	11
	2.3 Stakeholders	12
	2.4 Supporting Data/Research	12
Chapter 3:	Solution Design and Implementation	13
	3.1 Development and Design Process	13
	3.2 Tools and Technologies Used	13
	3.3 Solution Overview	14
	3.4 Engineering Standards Applied	14
	3.5 Solution Justification	15
Chapter 4:	Results and Recommendations	16

Chapter	Contents	Page No.
	4.1 Evaluation of Results	16
	4.2 Challenges Encountered	16
	4.3 Possible Improvements	17
	4.4 Recommendations	17
Chapter 5:	Reflection on Learning and Personal Development	18
	5.1 Key Learning Outcomes	18
	5.2 Challenges Encountered and Overcome	18
	5.3 Application of Engineering Standards	19
	5.4 Insights into the Industry	19
	5.5 Conclusion of Personal Development	20
Chapter 6:	Conclusion	21-22
	References	23
	Appendices	25

List of Tables

S.no	TOPIC	Page.no
1	Stakeholders	12
2	Tools and Technologies Used	13

CHAPTER 1

INTRODUCTION

1.1 Background Information

Application Programming Interfaces (APIs) have become an essential component of modern software systems. Web applications, mobile applications, cloud platforms, microservices, and distributed systems use APIs to exchange information and provide services.

Since APIs directly expose application functionality and data, they are frequently targeted by attackers. Attackers may attempt unauthorized access, credential attacks, injection attacks, excessive requests, token misuse, parameter manipulation, and exploitation of insecure business logic.

The **Intelligent API Attack Detection and Runtime Protection Framework** addresses this problem by continuously monitoring API traffic and analysing request behaviour. The framework combines rule-based detection, anomaly detection, behavioural analysis, and machine learning to identify potentially malicious API activity.

When an attack is detected, the runtime protection component can automatically respond by blocking the request, applying rate limits, restricting suspicious clients, or generating administrator alerts.

1.2 Project Objectives

The main objectives of this project are:

- To monitor API requests and responses in real time.
- To detect malicious API traffic and suspicious behaviour.
- To identify common API attacks such as unauthorized access, injection attempts, credential abuse, and excessive requests.
- To implement intelligent anomaly detection for identifying unusual API behaviour.
- To use machine learning techniques for improved attack detection.
- To provide runtime protection against detected attacks.

- To automatically block or restrict malicious requests.
- To implement rate limiting for preventing API abuse.
- To generate security alerts for administrators.
- To provide a centralized security monitoring dashboard.
- To store and analyse API security events.
- To reduce the response time to API security incidents.
- To improve API confidentiality, integrity, availability, and reliability.

1.3 Significance

API security is important because modern applications depend heavily on APIs for communication and data exchange. A successful API attack can result in unauthorized data access, account compromise, service disruption, data manipulation, and financial or operational losses. The proposed framework provides continuous monitoring instead of depending only on periodic security testing. It can identify suspicious API behaviour while the application is running.

The intelligent detection mechanism can identify both known attack patterns and unusual behaviour. Runtime protection allows the system to respond immediately when a high-risk request is detected. The centralized dashboard also provides security administrators with better visibility into API traffic, detected attacks, affected endpoints, and response actions.

1.4 Scope

The scope of this project includes:

- API traffic monitoring.
- REST API security.
- User authentication.
- JWT/OAuth-based authentication.
- Authorization and access control.
- API request and response logging.
- Attack pattern detection.

- Anomaly detection.
- Machine-learning-based attack detection.
- Rate limiting.
- Runtime request blocking.
- Suspicious client identification.
- Security alerts.
- Security event management.
- Administrator dashboard.
- API security analytics.
- Attack history and reporting.
- Performance and security testing.

Future enhancements may include advanced deep-learning models, automated threat intelligence integration, SIEM integration, distributed deployment, and adaptive security policies.

1.5 Methodology Overview

The project follows an iterative software development methodology consisting of:

1. Requirement Analysis
2. API Threat Identification
3. Threat Modelling
4. System Architecture Design
5. Database Design
6. API Monitoring Module Development
7. Attack Detection Module Development
8. Machine Learning Model Development
9. Runtime Protection Development

10. Security Dashboard Development
11. Integration and Testing
12. Performance Evaluation
13. Security Validation
14. Deployment
15. Maintenance and Future Enhancement

Each development phase is tested and evaluated to ensure that the security framework operates correctly and efficiently.

CHAPTER 2

PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Description of the Problem

APIs are widely used to provide access to application functionality and data. However, insecure APIs can provide attackers with direct access to sensitive resources. Attackers may

exploit weak authentication, insufficient authorization, insecure input validation, excessive API requests, predictable object identifiers, and poorly implemented business logic.

For example, an attacker may repeatedly send login requests to guess credentials. Similarly, an attacker may send a large number of requests to a particular endpoint to consume server resources. Traditional security systems may identify known attack patterns but may have difficulty identifying unusual behaviour that does not match predefined signatures.

Another problem is the large volume of API traffic generated by modern applications. Manually analysing every request is impractical.

2.2 Evidence of the Problem

Examples include:

- Repeated failed authentication requests.
- Brute-force login attempts.
- Unauthorized access to API resources.
- Excessive API requests.
- Injection attempts through API parameters.
- Manipulation of request data.
- Token misuse.
- Abnormal access to multiple objects.
- Automated API enumeration.
- Denial-of-service-like request patterns.

For example, if a normal user accesses an API 20 times within several minutes but another client generates thousands of requests within the same period, the second behaviour may indicate automated abuse.

2.3 Stakeholders

The main stakeholders of the proposed framework include:

Security Administrators

Monitor API attacks, investigate security events, configure policies, and manage runtime protection.

API Developers

Use security information to identify and correct vulnerabilities in API implementations.

System Administrators

Manage servers, databases, deployment environments, and system availability.

Application Users

Benefit from secure, reliable, and continuously protected API services.

Organizations

Require protection of sensitive data, business services, user accounts, and API infrastructure.

Incident Response Teams

Use security alerts and event information to investigate and respond to attacks.

2.4 Supporting Data / Research

The proposed framework is based on important API security concepts such as authentication, authorization, input validation, rate limiting, secure logging, least privilege, anomaly detection, and runtime monitoring.

API security guidelines identify several important risks, including broken authentication, broken authorization, unrestricted resource consumption, security misconfiguration, and injection-related vulnerabilities.

Traditional rule-based systems are effective at detecting known attack patterns. However, attackers can modify their behaviour to avoid predefined rules.

Machine learning and anomaly detection can complement traditional security mechanisms by learning normal API behaviour and identifying deviations.

Therefore, the proposed framework combines multiple security mechanisms rather than depending on a single detection technique.

CHAPTER 3

SOLUTION DESIGN AND IMPLEMENTATION

3.1 Development and Design Process

The proposed framework follows a modular architecture consisting of multiple interconnected security modules.

The API client sends a request to the API protection layer. The monitoring module collects security-relevant information from the request.

The authentication and authorization mechanisms verify whether the client has permission to access the requested API.

The intelligent detection engine then analyses the request using predefined security rules and behavioural characteristics.

If suspicious behaviour is identified, a risk score is generated. The runtime protection module uses this risk level to determine the appropriate response.

3.2 Tools and Technologies Used

Technology	Purpose
HTML	Dashboard structure
CSS	Dashboard styling
JavaScript	Frontend interaction
React	Security dashboard development
Python	Backend and security processing
Flask/FastAPI	API development
MySQL/Firebase	Security event storage

Technology	Purpose
JWT	Authentication
OAuth 2.0	Authorization
Machine Learning	Intelligent attack detection
Git	Version control
GitHub	Project management
Visual Studio Code	Development environment

3.3 Solution Overview

The proposed framework consists of the following major modules:

1. API Traffic Monitoring Module
2. Authentication and Authorization Module
3. Intelligent Attack Detection Module
4. Runtime Protection & Response Module
5. Security Analytics Dashboard
6. Security Data Management Module

The system also provides map-based visualization, which helps administrators identify the locations of multiple reported issues. This makes it easier to manage complaints and plan maintenance activities efficiently.

3.4 Engineering Standards Applied

The proposed framework follows important software and security engineering principles:

- Modular architecture.
- Secure coding practices.
- Input validation.
- Authentication and authorization.
- Least privilege.
- Defence in depth.

- Secure logging.
- Error handling.
- Secure configuration.
- Database security.
- Version control.
- Testing and validation.
- Security monitoring.
- Separation of responsibilities.

The system should also follow established API security practices and secure software development principles.

3.5 Solution Justification

The proposed framework is necessary because traditional API protection mechanisms may not detect every type of attack.

Rule-based systems are effective for known attacks but may not identify new or modified attack techniques.

Anomaly detection can identify unusual behaviour but may produce false positives.

Runtime protection can immediately contain attacks but depends on reliable detection.

By combining these technologies, the proposed framework provides a layered security approach.

The combination of monitoring, intelligent detection, and runtime protection can improve API visibility, attack detection, incident response, and overall system security.

CHAPTER 4

RESULTS AND RECOMMENDATIONS

4.1 Evaluation of Results

The developed Intelligent API Attack Detection and Runtime Protection Framework was designed and tested to verify its major security functions. The system successfully monitors API requests and responses and records important security-related information such as API endpoints, HTTP methods, response status, authentication results, request frequency, and suspicious activity patterns. The monitoring module provides continuous visibility into API traffic and helps identify abnormal behaviour.

The Intelligent Attack Detection Module analyses API traffic using predefined security rules and behavioural analysis. The system is designed to detect suspicious activities such as repeated authentication failures, excessive API requests, unauthorized access attempts, injection patterns, and unusual API usage. The anomaly detection component helps identify behaviour that differs from normal API traffic.

The Runtime Protection and Response Module provides automatic security actions when suspicious activity is detected. Depending on the severity of the detected activity, the system can generate alerts, apply rate limiting, temporarily restrict suspicious clients, or block malicious requests. This reduces the time required to respond to security incidents.

The administrator dashboard provides a centralized view of API traffic and security events. Administrators can monitor detected attacks, affected endpoints, severity levels, risk scores, and response actions. The system therefore improves API visibility and provides a structured method for analysing and responding to security threats.

4.2 Challenges Encountered

Several challenges were encountered during the development of the Intelligent API Attack Detection and Runtime Protection Framework. One major challenge was distinguishing between legitimate high-volume API traffic and malicious requests. Some applications may naturally generate a large number of API requests, making it difficult to determine whether the activity is normal or represents an attack.

Another challenge was reducing false positives in the attack detection process. An intelligent detection system may sometimes classify legitimate API behaviour as suspicious. Proper

threshold configuration, behavioural analysis, and continuous testing are therefore required to improve detection accuracy.

Maintaining low response time was also an important challenge because the security framework analyses API requests during runtime. The protection mechanism must detect threats without introducing significant delays to normal API operations.

Another challenge was protecting security logs and monitoring data. The system may process sensitive information during API monitoring, so appropriate security controls must be applied to prevent unauthorized access to security records.

These challenges were addressed through modular system design, continuous testing, validation of detection rules, careful configuration of security policies, and controlled testing using authorized API traffic.

4.3 Possible Improvements

The framework can be improved further by introducing advanced machine-learning and deep-learning techniques for detecting complex API attack patterns. Endpoint-specific behavioural models can also be developed so that the system can understand the normal behaviour of individual API endpoints.

Adaptive security thresholds can be introduced to automatically adjust detection levels according to changes in API traffic. The framework can also be integrated with threat intelligence sources to improve the identification of known malicious activities.

Another possible improvement is the integration of the framework with Security Information and Event Management systems. This would allow security events generated by the API protection framework to be correlated with other security events within an organization.

The system can also be enhanced with explainable artificial intelligence so that administrators can understand why a particular API request was classified as suspicious. Automated incident-response features can further improve the ability of the system to respond quickly to high-severity attacks.

4.4 Recommendations

It is recommended that strong authentication and authorization mechanisms be implemented for all protected APIs. API security policies should be regularly reviewed and updated according to new threats and changes in application behaviour.

Security logs should be protected from unauthorized access and sensitive information such as passwords and complete authentication tokens should not be stored unnecessarily. Administrator access to the security dashboard should also be protected using strong authentication and appropriate access controls.

The automated blocking mechanism should be carefully configured to reduce the possibility of blocking legitimate users. Detection thresholds should be continuously reviewed using both normal API traffic and authorized security-testing traffic.

Regular security testing, vulnerability assessment, monitoring, and maintenance should be performed to ensure that the framework continues to provide effective API protection.

Chapter 5

Reflection On Learning And Personal Development

5.1 Key Learning Outcomes

The development of the Intelligent API Attack Detection and Runtime Protection Framework provided valuable practical knowledge in API security and cybersecurity. The project helped in understanding how APIs communicate with frontend applications, backend services, databases, and other systems.

The project provided practical experience in authentication, authorization, JWT, OAuth, API monitoring, logging, attack detection, anomaly detection, machine learning, and runtime

protection. It also improved understanding of how security mechanisms can be integrated into an application without affecting normal functionality.

Working on the project improved programming skills, debugging ability, analytical thinking, problem-solving skills, teamwork, communication, and documentation. The project also helped connect theoretical cybersecurity concepts with a practical API security solution.

5.2 Challenges Encountered and Overcome

During the development of the project, several technical challenges were encountered. One of the major challenges was understanding different types of API attacks and determining how their behaviour differs from normal API traffic.

Another challenge was developing detection rules that could identify suspicious activities without generating a large number of false alerts. This required repeated testing and adjustment of detection conditions.

Integrating intelligent detection with runtime protection was also challenging because the system needs to analyse API requests and respond quickly without significantly affecting application performance.

These challenges were overcome through continuous learning, testing, debugging, and improvement of the system architecture. The experience helped develop a systematic approach to solving technical and security-related problems.

5.3 Application of Engineering Standards

Software engineering and cybersecurity principles were applied throughout the development of the framework. The system was divided into separate modules to improve maintainability, scalability, and future modification.

Secure coding practices were considered during API development. Authentication and authorization mechanisms were used to control access to protected resources. Input validation and secure logging practices were also considered to reduce security risks.

Testing was performed on important modules such as API monitoring, authentication, attack detection, runtime protection, alert generation, and dashboard functionality. Version control and proper documentation were also considered to maintain an organized project structure.

5.4 Insights into the Industry

This project provided an understanding of the increasing importance of API security in modern software applications. Many organizations use APIs for mobile applications, cloud services, microservices, online platforms, and third-party integrations.

The project demonstrated that API security requires continuous monitoring rather than relying only on security testing performed during application development. Real-time attack detection and automated runtime protection can help organizations respond more quickly to security incidents.

The project also provided an understanding of how artificial intelligence and machine learning can support cybersecurity. Intelligent detection can identify unusual API behaviour and complement traditional security mechanisms such as authentication, authorization, and rate limiting.

5.5 Conclusion of Personal Development

Working on this project resulted in significant technical and personal development. The project improved confidence in designing and developing cybersecurity solutions for real-world applications.

It strengthened knowledge of API security, programming, databases, machine learning, security monitoring, and runtime protection. It also improved analytical thinking, teamwork, communication, debugging, and project management skills.

The experience provided practical knowledge about integrating multiple technologies into a single security framework. Overall, the project contributed to professional development and provided useful experience for future work in cybersecurity, API security, and secure software development.

growth and provided useful experience for future work in software development and technology-based public service applications.

CHAPTER 6

CONCLUSION

The Intelligent API Attack Detection and Runtime Protection Framework provides a practical security solution for monitoring and protecting modern API-based applications. The framework continuously monitors API traffic and analyses requests to identify suspicious activities and potential security threats.

The system combines API traffic monitoring, authentication and authorization, rule-based attack detection, behavioural analysis, anomaly detection, and runtime protection. This layered approach helps identify both known attack patterns and unusual API behaviour.

The Runtime Protection and Response Module provides automatic actions when suspicious activity is detected. Depending on the severity of the threat, the system can generate alerts, apply rate limiting, restrict suspicious clients, or block malicious requests. This helps reduce the time between attack detection and security response.

The security dashboard provides administrators with centralized information about API traffic, detected attacks, affected endpoints, severity levels, and response actions. This improves security visibility and supports faster investigation of API-related incidents.

The project demonstrates that combining traditional API security controls with intelligent detection and automated runtime protection can improve the security, reliability, and availability of API-driven applications. The framework can be further enhanced with advanced machine-learning models, deep learning, threat intelligence, SIEM integration, adaptive detection, and automated incident response.

In conclusion, the proposed Intelligent API Attack Detection and Runtime Protection Framework provides an effective foundation for protecting API-based applications against evolving cyber threats and supports organizations in achieving continuous API security and faster runtime response.

REFERENCES

1. OWASP Foundation. *OWASP API Security Top 10*. OWASP Foundation.
2. OWASP Foundation. *OWASP Application Security Verification Standard (ASVS)*. OWASP Foundation.
3. OWASP Foundation. *OWASP Secure Coding Practices*. OWASP Foundation.
4. National Institute of Standards and Technology (NIST). *Cybersecurity Framework (CSF)*. U.S. Department of Commerce.
5. National Institute of Standards and Technology (NIST). *Computer Security Incident Handling Guide, SP 800-61*. NIST.
6. Jones, M., Bradley, J., and Sakimura, N. *JSON Web Token (JWT), RFC 7519*. Internet Engineering Task Force (IETF).
7. Hardt, D. *The OAuth 2.0 Authorization Framework, RFC 6749*. Internet Engineering Task Force (IETF).
8. Jones, M. and others. *The OAuth 2.0 Authorization Framework: Bearer Token Usage, RFC 6750*. Internet Engineering Task Force (IETF).
9. Sommerville, I. *Software Engineering*. Pearson Education.
10. Pressman, R. S., and Maxim, B. R. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
11. Mitchell, T. M. *Machine Learning*. McGraw-Hill.
12. Python Software Foundation. *Python Documentation*. Python Software Foundation.
13. FastAPI. *FastAPI Documentation*. FastAPI Official Documentation.
14. Pallets Projects. *Flask Documentation*. Flask Official Documentation.
15. React Team. *React Documentation*. React Official Documentation.